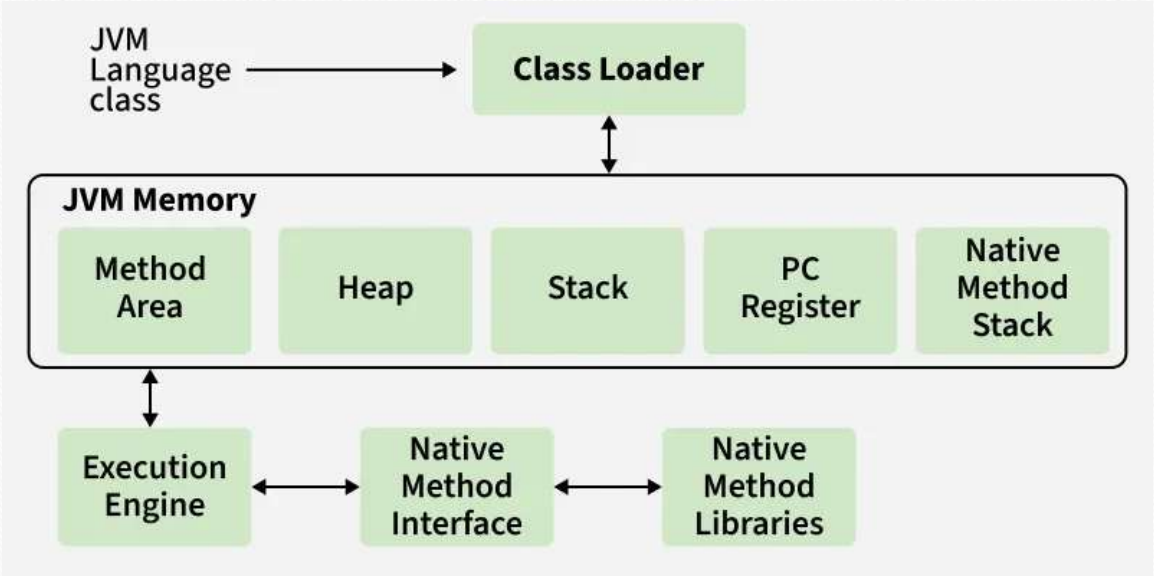
Classes and Objects

JVM

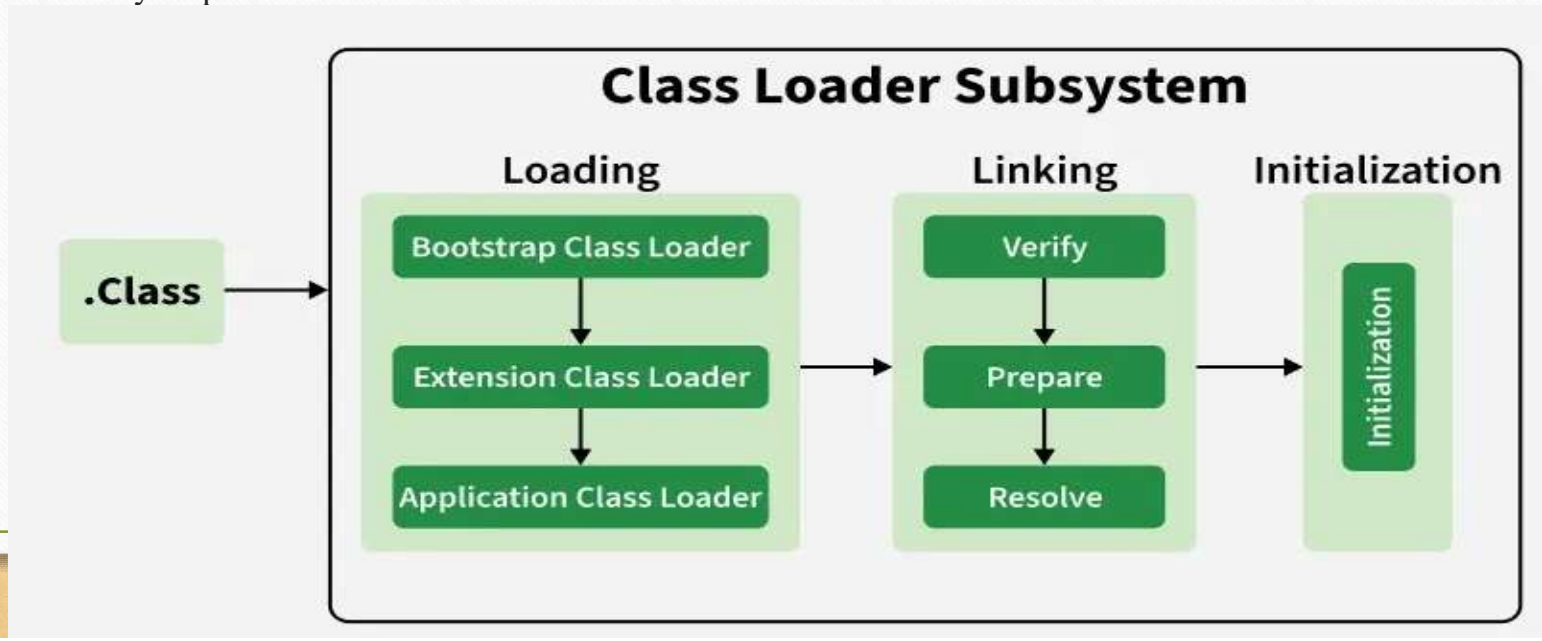
- The Java Virtual Machine (JVM) is a core component of the Java Runtime Environment (JRE) that allows Java programs to run on any platform without modification. JVM acts as an interpreter between Java bytecode and the underlying hardware, providing Java's famous Write Once, Run Anywhere (WORA) capability.
- Java source (.java) -> compiled by javac -> bytecode (.class)
- JVM loads the bytecode, verifies it, links it, and then executes it
- Execution may involve interpreting bytecode or using Just-In-Time (JIT) compilation to convert "hot" code into native machine code for performance
- Garbage collection runs in the background to reclaim memory from unused objects

Architecture of JVM



1. Class Loader Subsystem

It is mainly responsible for three activities.



. Loading

Reads .class files and stores class metadata in the Method Area.

Creates a Class object in the heap representing the loaded class.

---

2. Linking: Responsible for preparing the loaded class for execution. It includes three steps:

Verification: Ensures the bytecode follows JVM rules and is safe to execute.

Preparation: Allocates memory for static variables and assigns default values.

Resolution: Converts symbolic references into direct references in memory.

3. Initialization

Assigns actual values to static variables.

Executes static blocks defined in the class.

Class Loader Types

Bootstrap Class Loader: Loads core Java classes (JAVA_HOME/lib).

Extension Class Loader: Loads classes from extensions directory (JAVA_HOME/jre/lib/ext).

System/Application Class Loader: Loads classes from the application classpath.

2. JVM Memory Areas

Method Area: Stores class-level information like class name, parent class, methods, variables, and static data. Shared across the JVM.

Heap Area: Stores all objects. Shared across the JVM.

Stack Area: Each thread has its own runtime stack; stores method calls, local variables in stack frames. Destroyed when the thread ends.

PC Registers: Hold the address of the currently executing instruction for each thread.

Native Method Stacks: Each thread has a separate stack for native method execution.

3. Execution Engine

Execution engine executes the ".class" (bytecode). It reads the byte-code line by line, uses data and information present in various memory area and executes instructions. It can be classified into three parts:

Interpreter: It interprets the bytecode line by line and then executes. The disadvantage here is that when one method is called multiple times, every time interpretation is required.

Just-In-Time Compiler(JIT): It is used to increase the efficiency of an interpreter. It compiles the entire bytecode and changes it to native code so whenever the interpreter sees repeated method calls, JIT provides direct native code for that part so re-interpretation is not required, thus efficiency is improved.

Garbage Collector: It destroys un-referenced objects.

4. Java Native Interface (JNI)

It is an interface that interacts with the Native Method Libraries and provides the native libraries(C, C++) required for the execution. It enables JVM to call C/C++ libraries and to be called by C/C++ libraries which may be specific to hardware.

5. Native Method Libraries

These are collections of native libraries required for executing native methods. They include libraries written in languages like C and C++

Object Lifetime in:

Object lifetime refers to the period between an object's creation (instantiation) and its destruction (reclamation of memory). Unlike C/C++, Java manages this automatically through the JVM.1.2

Stages of an Object's Life: CreationMemory allocated on the heap using the new keyword.Constructor runs, fields are initialized.

Student s = new Student("Amit");

s = null; // object Amit becomes eligible for GC (if no other references exist)

JVM's Garbage Collector (GC) identifies unreachable objects and reclaims their memory. Timing is not guaranteed the developer cannot force immediate collection.

Garbage Collection

An automatic memory management process performed by the JVM that reclaims heap memory occupied by objects that are no longer reachable by the running program.

It prevents memory leaks (to a large extent). Removes need for manual free()/delete() as in C/C++.Reduces dangling pointer errors.

OOPs

Object-oriented programming (OOP) involves programming using objects.

An *object* represents an entity in the real world that can be distinctly identified.

For example, a student, a desk, a circle, a button, and even a loan can all be viewed as objects.

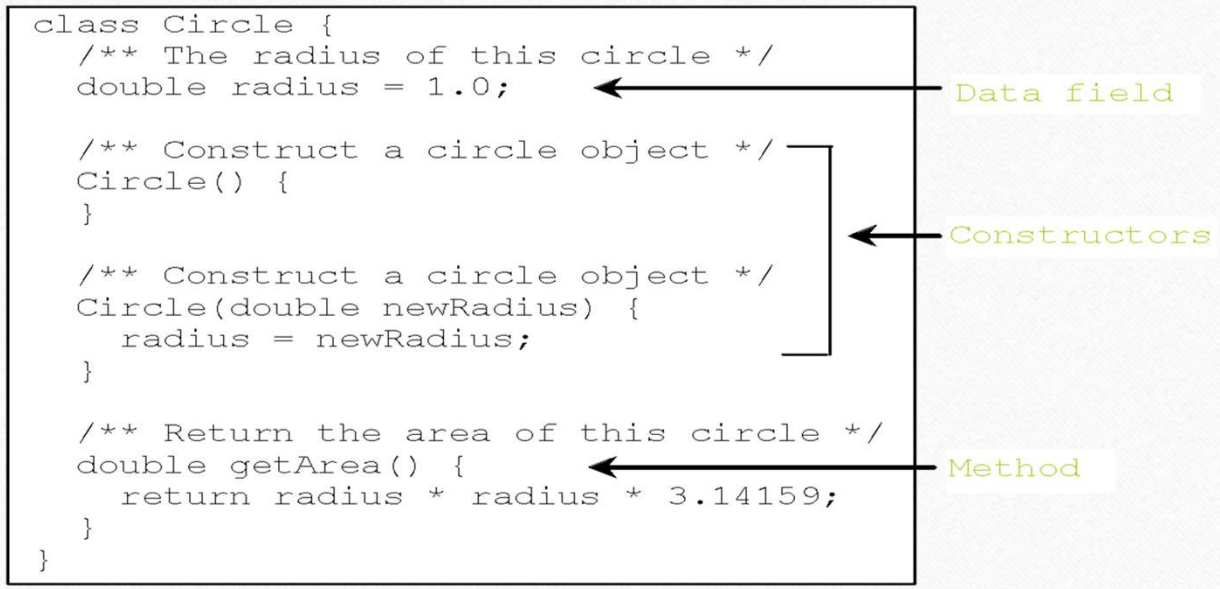
Classes

Classes are constructs that define objects of the same type. A Java class uses variables to define data fields and methods to define behaviors.

Additionally, a class provides a special type of methods, known as constructors, which are invoked to construct objects from the class.

Syntax:

```
Class classname
{ fields declaration:
  methods declaration;
}
```


class Circle {
/** The radius of this circle */
double radius = 1.0;
/** Construct a circle object */
Circle() {
}
/** Construct a circle object */
Circle(double newRadius) {
radius = newRadius;
}
/** Return the area of this circle */
double getArea() {
return radius * radius * 3.14159;
}
}
Data field
Constructors
Method

An object has a unique identity, state, and behavior.

The *state* of an object consists of a set of *data fields* (also known as *properties*) with their current values.

The *behavior* of an object is defined by a set of methods.

An object has both a state and behavior. The state defines the object, and the behavior defines what the object does.

- When objects are created, the initial value of data fields is unknown unless its users explicitly do so. For example,
  - ObjectName.DataField1 = 0; // OR
  - ObjectName.SetDataField1(0);
- In many cases, it makes sense if this initialisation can be carried out by default without the users explicitly initialising them.
  - For example, if you create an object of the class called "Counter", it is natural to assume that the counter record-keeping field is initialised to zero unless otherwise specified differently.

```
class Counter
{
        int CounterIndex;
        …}
Counter counter1 = new Counter();
```

  - What is the value of "counter1.CounterIndex" ?
- In Java, this can be achieved though a mechanism called constructors.

Constructor

Constructor is a block of codes similar to the method. It is called when an instance of the class is created. At the time of calling constructor, memory for the object is allocated in the memory.

It is a special type of method which is used to initialize the object.

Every time an object is created using the new() keyword, at least one constructor is called.

It calls a default constructor if there is no constructor available in the class. In such case, Java compiler provides a default constructor by default.

Rules for creating Java constructor

There are two rules defined for the constructor.

1. Constructor name must be the same as its class name
2. A Constructor must have no explicit return type not even void.
3. A Java constructor cannot be abstract, static, final, and synchronized

Types of Java constructors

There are two types of constructors in Java:

1. Default constructor (no-arg constructor)
2. Parameterized constructor

Default Constructor

A constructor is called "Default Constructor" when it doesn't have any parameter.

The default constructor is used to provide the default values to the object like 0, null, etc., depending on the type.

Syntax of default constructor:

```
<class_name>()
{}
```

Note: If there is no constructor in a class, compiler automatically creates a default constructor.

Parameterized Constructor

A constructor which has a specific number of parameters is called a parameterized constructor.

The parameterized constructor is used to provide different values to distinct objects. However, you can provide the same values also.

```
// A class representing a Student to demonstrate constructor overloading
class Student {
  // Instance variables
   String name;    int rollNumber;
 // 1. Default Constructor (No-argument constructor)    // Used to initialize objects with default fallback values
   public Student() {
this.name = "Unknown";
this.rollNumber = 0;
 System.out.println("Default Constructor Called.");    }
```

```
// 2. Parameterized Constructor    // Used to initialize objects with custom user-provided values
public Student(String studentName, int studentRollNumber)
{       this.name = studentName;
this.rollNumber = studentRollNumber;
System.out.println("Parameterized Constructor Called.");    }
  // Method to display student details
 public void displayDetails() {       System.out.println("Name: " + name + ", Roll Number: " + rollNumber);       }}
```

```
// Main class to execute the program
public class Main {    public static void main(String[] args)
{       // Creating an object using the Default Constructor
System.out.println("Creating student1:");
    Student student1 = new Student();
   student1.displayDetails();       // Creating an object using the Parameterized
Constructor
 System.out.println("Creating student2:");
 Student student2 = new Student("ABC", 101);
  student2.displayDetails();    }}
```

The New *this* keyword

this keyword can be used to refer to the object itself. It is generally used for accessing class members (from its own methods) when they have the same name as those passed as arguments.

```
public class Circle {
   public double x,y,r;

// Constructor
   public Circle (double x, double y, double r) {
        this.x = x;
        this.y = y;
        this.r = r;
   }
 //Methods to return circumference and area
}
```

Static Members

- Java supports definition of global methods and variables that can be accessed without creating objects of a class. Such members are called Static members.
- Define a variable by marking with the static methods.
- This feature is useful when we want to create a variable common to all instances of a class.
- One of the most common example is to have a variable that could keep a count of how many objects of a class have been created.
- Note: Java creates only one copy for a static variable which can be used even if the class is never instantiated.

Static Variables - Example

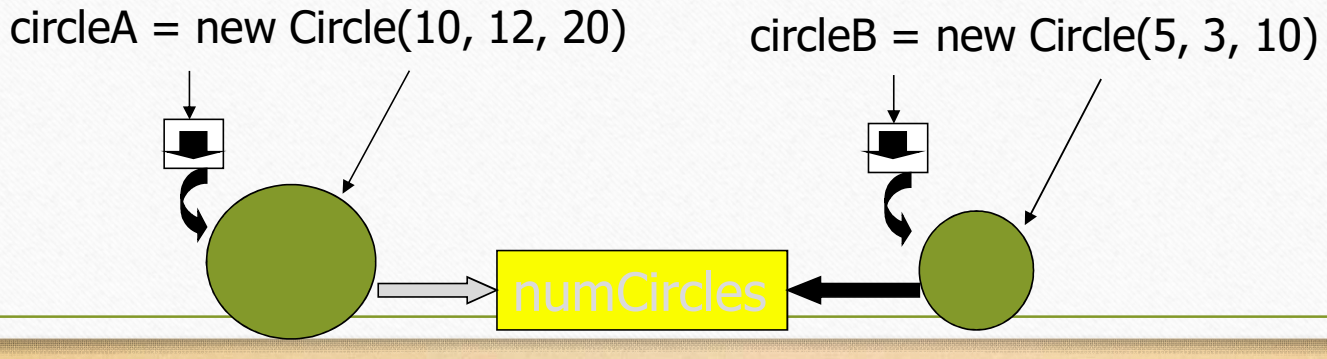
Using *static variables:*

```
public class Circle {
      // class variable, one for the Circle class, how many circles
      private static int numCircles = 0;
      private double x,y,r;

      // Constructors...
      Circle (double x, double y, double r){
            this.x = x;
            this.y = y;
            this.r = r;
            numCircles++;
      }   }
```

Class Variables - Example

- Using *static variables:*

```
public class CountCircles {

        public static void main(String args[]){
            Circle circleA = new Circle( 10, 12, 20);    // numCircles = 1
            Circle circleB = new Circle( 5, 3, 10);      // numCircles = 2
    }
}
```



Instance versus Static Variables

- Instance variables : One copy per object. Every object has its own instance variable.
  - E.g. x, y, r (centre and radius in the circle)
- Static variables : One copy per class.
  - E.g. numCircles (total number of circle objects created)

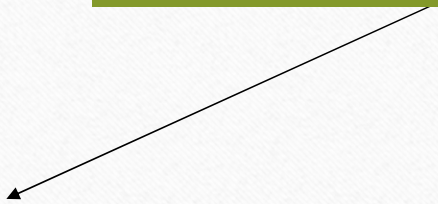
Static Methods

- A class can have methods that are defined as static (e.g., main method).
- Static methods can be accessed without using objects. Also, there is NO need to create objects.
- They are prefixed with keyword "static"
- Static methods are generally used to group related library functions that don't depend on data members of its class. For example, Math library functions.

Comparator class with Static methods

```
// Comparator.java: A class with static data items comparison methods
class Comparator {
      public static int max(int a, int b)
      {     if( a > b)
                  return a;
            else
                 return b;}
            public static String max(String a, String b)
      {   if( a.compareTo (b) > 0)
                  return a;
            else
                  return b;}
      }
class MyClass {
```

Directly accessed using ClassName (NO Objects)



```
public static void main(String args[])
 {
      String s1 = "Melbourne";
      String s2 = "Sydney";
      String s3  = "Adelaide";

      int a = 10;
      int b = 20;

      System.out.println(Comparator.max(a, b)); // which number is big
      System.out.println(Comparator.max(s1, s2)); // which city is big
      System.out.println(Comparator.max(s1, s3)); // which city is big
 }
}
```

Static methods restrictions

- They can only call other static methods.
- They can only access static data.
- They cannot refer to “this” or “super” (more later) in anyway.

Inner Class

Inner Classes in Java

An **inner class** is a class defined within another class. Used to logically group classes used in one place, increase encapsulation, and produce more readable/maintainable code.

Nested Classes

- Static Nested Class
- Non-static Inner Classes
  - Member Inner Class
  - Local Inner Class
  - Anonymous Inner Class

- **Member (Regular) Inner Class**
- Declared inside a class, outside any method.
- Has access to all members (including private) of the outer class.
- Requires an instance of the outer class to be instantiated.

```
class Outer {
    private int data = 10;

    class Inner {
        void display() {
            System.out.println("Outer data: " + data);}}}

public class Main {
    public static void main(String[] args) {
        Outer outer = new Outer();
        Outer.Inner inner = outer.new Inner();  // note the syntax
        inner.display();}}
```

Why Use Inner Classes?

Logical grouping: If a class is useful to only one other class, embed it there.

Increased encapsulation: Inner class can access outer class's private members; outside classes cannot access the inner class directly (for member/local classes).

More readable and maintainable code: Keeps helper classes close to where they're used.

Event handling / callbacks: Anonymous classes are convenient for one-off implementations.

Circle Program

```
// Circle.java:  Contains both Circle class and its user class
//Add Circle class code here
class MyMain
{
      public static void main(String args[])
      {
            Circle aCircle;  // creating reference
            aCircle = new Circle(); // creating object
            aCircle.x = 10;  // assigning value to data field
            aCircle.y = 20;
            aCircle.r = 5;
            double area = aCircle.area(); // invoking method
            double circumf = aCircle.circumference();
            System.out.println("Radius="+aCircle.r+" Area="+area);
            System.out.println("Radius="+aCircle.r+" Circumference ="+circumf);
      }
}
```

```
java MyMain
Radius=5.0 Area=78.5
Radius=5.0 Circumference =31.400000000000002
```